

Document Number: P0658R1

Date: 2017-06-11

Audience: Evolution Working Group

Reply-to: Christopher Di Bella christopher@codeplay.com

Proposal for adding alias declarations to concepts

Abstract

This document proposes to add *alias-declarations* to *requirement-bodies* so that types may be renamed without the need for creating exposition-only concepts that have additional template typenames for convenience and readability.

Table of contents

- [Primary motivation](#)
 - [Motivation 1](#)
 - [Motivation 2](#)
- [Proposals](#)
 - [Proposal 1](#)
 - [Proposal 2](#)
 - [Proposal 3](#)
- [Conclusion](#)
- [Acknowledgements](#)

Disclaimer

This document uses diff syntax. Lines that are prefixed with a `+` are highlighted green to indicate a proposed addition to the core language.

Primary motivation

Motivation 1

Concepts are fun to work with, and fun to design, but anything with more than a few names is tedious to type up, and tedious to read. Consider this rather verbose attempt at drafting a concept for a sequence container:

```
// using value_type_t, difference_type_t, and concepts from Ranges TS
template <typename X, typename... Args>
concept bool SequenceContainer = Container<X> && requires {
    typename value_type_t<X>;
    typename difference_type_t<X>;
    requires Constructible<X, difference_type_t<X>, value_type_t<X>>();
    requires Constructible<X, initializer_list<value_type_t<X>>>();
    requires Assignable<X&, initializer_list<value_type_t<X>>>();
    // ...
};
```

Our current `SequenceContainer` is quite jarring to read, so we might like to split it into `SequenceContainer` and `detail::SequenceConstructible`:

```
namespace detail {
    template <typename X, typename T, typename D, typename... Args>
    concept bool SequenceConstructible = Constructible<X, D, T>() &&
        Constructible<X, initializer_list<T>>() &&
        Assignable<X&, initializer_list<T>>()
        // ...
    ;
    // Other SequenceContainer details here...
} // namespace detail

template <typename X, typename... Args>
concept bool SequenceContainer = Container<X> &&
    detail::SequenceConstructible<X, value_type_t<X>, difference_type_t<X>, Args...> /
    // ...
;
```

Now we can read the implementation of a `SequenceContainer` without the added noise of `value_type_t<X>`, `difference_type_t<X>`, and `initializer_list<value_type_t<X>>` at every turn, but we've now created a second, exposition-only concept with exactly one use-case: to facilitate the readability of our usable concept that (hopefully!) has many use-cases. Furthermore, we need to find a meaningful name for our exposition-only concepts so that they are easy to understand.

Desired outcome

It would be nice to have something that resembles:

```
template <typename X, typename... Args>
concept bool SequenceContainer = Container<X> && requires {
+   requires using value_type = value_type_t<X>; // type alias value_type, typename r
+   requires using difference_type = difference_type_t<X>;
+   using initializer_list_type = initializer_list<value_type>; // just a type alias

    requires Constructible<X, difference_type, value_type>();
    requires Constructible<X, initializer_list_type>();
    requires Assignable<X&, initializer_list_type>();
    // ...
};
```

Why is this desirable, given the above motivation?

Exposition-only concepts that exist solely to improve readability are removed.

Motivation 2

Let's now consider an attempt to turn `std::inner_product` into a concept-checked `inner_product`:

```
template <InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
CopyConstructible T,
typename Proj1 = identity, typename Proj2 = identity,
IndirectRegularInvocable<projected<I1, Proj1>,
    projected<I2, Proj2>> Op2 = multiplies<>,
RegularInvocable<T,
    indirect_result_of_t<Op2&(projected<I1, Proj1>,
        projected<I2, Proj2>>>> Op1 = plus<>>
requires
    Assignable<T&, const T&>() &&
    Assignable<T&, result_of_t<Op1&(T,
        indirect_result_of_t<Op2&(projected<I1, Proj1>, projected<I2, Proj2>>>>())
T inner_product(I1 first1, S1 last1, I2 first2, S2 last2, T init, Op1 op1 = Op1{},
    Op2 op2 = Op2{}, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

Ugly. There's lots of repetition, and overall, it's quite difficult to reason if it's correct. Let's tidy that up a bit:

```

template <InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
CopyConstructible T,
typename Op1 = plus<>, typename Op2 = multiplies,
typename Proj1 = identity, typename Proj2 = identity,
typename Arg1 = projected<I1, Proj1>, typename Arg2 = projected<I2, Proj2>,
typename InnerResult = indirect_result_of_t<Op2&(Arg1, Arg2)>,
typename OuterResult = result_of_t<Op1&(T, InnerResult)>>
requires
    Assignable<T&, const T&>() &&
    IndirectRegularInvocable<Op2, Arg1, Arg2>() &&
    RegularInvocable<Op1, T, InnerResult>() &&
    Assignable<T&, OuterResult>()
T inner_product(I1 first1, S1 last1, I2 first2, S2 last2, T init, Op1 op1 = Op1{},
Op2 op2 = Op2{}, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

```

That's much easier on the eyes, and our operations are properly ordered! Sadly, we've introduced an opportunity for Crafty Programmer who thinks that they know better than their compiler (and library specification) to manipulate the template specialisation to allow syntactically-compatible-but-semantically-incompatible types to pass the concept check. This is an unlikely scenario, but we can't rule it out from happening for all concept-checked functions.

Desired outcome

It'd be great if we could do this instead:

```

template <InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
CopyConstructible T, typename Op1 = plus<>, typename Op2 = multiplies<>,
typename Proj1 = identity, typename Proj2 = identity>
requires requires {
    requires Assignable<T&, const T&>();

+   using Arg1 = projected<I1, Proj1>;
+   using Arg2 = projected<I2, Proj2>;
+   using Inner_result = indirect_result_of_t<Op2&(Arg1, Arg2)>;
+   using Outer_result = result_of_t<Op1&(T, InnerResult)>;

    requires IndirectRegularInvocable<Op2, Arg1, Arg2>();
    requires RegularInvocable<Op1, T, InnerResult>();
    requires Assignable<T&, OuterResult>();
}
T inner_product(I1 first1, S1 last1, I2 first2, S2 last2, T init, Op1 op1 = Op1{},
Op2 op2 = Op2{}, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

```

Why is this desirable, given the above motivation?

Unlike the example given in our motivation, this indisputably communicates to both programmers and the compiler that `Arg1`, `Arg2`, `Inner_result` and `Outer_result` are not supposed to be changed by removing them from a position where a programmer can accidentally or deliberately provide an alternative type.

Proposals

This document suggests three compounding proposals, provides minor motivations for each, and then discusses their benefits and drawbacks.

Proposal 1

A requirements-body allows alias-declarations.

```
// Example 1
template <class T>
requires(T t) {
    typename value_type_t<T>;
+   using value_type = value_type_t<T>;
    {t + t} -> value_type;
}
void foo(T t) {}

// Example 2
template <class T>
requires(T t) {
    typename value_type_t<T>;
+   using value_type = value_type_t<T>;
    {t + t} -> value_type;
}
void foo(T t)
{
    value_type bar{}; // ill-formed, value_type not in scope
}

// Example 3
template <class T>
requires(T t) {
    typename value_type_t<T>;
+   using value_type = value_type_t<T>;
    {t + t} -> value_type;
}
void foo(T t)
{
    using value_type = value_type_t<T>;
```

```
value_type bar{}; // okay, value_type in scope
}
```

Relevant motivation

The key motivation for this proposal is its simplicity: an alias declared inside a *requirement-body* stays inside said *requirement-body*. Names are guaranteed not to leak into other namespaces, including function definitions and class specifications.

However, the author feels that this proposal, while simple, is very short sighted. A programmer that is specifying the requirements for a function or class is probably also writing said code. This proposal may simplify the verbosity of a requirements-body, but the programmer still needs to explicitly type out the type requirement and a second type alias in the function/class definition. Forcing the programmer to write out the same type (with the same name) multiple times is potentially error prone. We explore solutions to these in Proposals 2 and 3.

Implementation status

This proposal is currently being implemented using a branch from GCC trunk.

At present, it is possible to declare an alias inside the *requirement-body*, but the alias leaks outside of the enclosing body. This means that it isn't possible to write such code:

```
template <typename T>
concept bool Foo = requires {
    using Bar = double;
};

using Bar = int; // error: Bar already declared on line 3
```

This is under investigation, and can be found on [GitHub](#).

Proposal 2

Proposal 2 complements Proposal 1 by extending aliases declared inside the *requirement-body* to the function or class definition immediately following the body.

```
// Example 1
template <class T>
requires(T t) {
    typename value_type_t<T>;
    using value_type = value_type_t<T>;
    {t + t} -> value_type;
```

```

}
void foo(T t)
{
    value_type bar{}; // ok, value_type in scope
}

// Example 2
template <class T>
concept bool SequenceContainer = Container<X> && requires {
    typename value_type_t<X>;
    using value_type = value_type_t<X>;
    ...
};
template <SequenceContainer T>
struct Matrix {
    Matrix(initializer_list<value_type> l); // ill-formed: value_type not in scope
};

```

Relevant motivation

This is a nice addition to the original proposal, as it allows for names declared inside a *requirement-body* to be declared only once. Names are still contained outside of the whole function/class scope, so there is no global pollution. This would be great in an associative container environment, for example:

```

template <class Key, class Value, class Compare, class A>
requires requires {
    using key_type = Key;
    using mapped_type = Value;
    using value_type = pair<key_type, mapped_type>;
    requires Allocator<A, value_type>;
}
class map {
    // value_type, etc. already declared
};

```

The author foresees two drawbacks in this proposal:

1. Aliases convenient for the *requirement-body* but unused outside are available for accidental misuse. This is not any different to using an exposition-only template parameter, but it is a point worth considering. This could potentially be resolved by making all aliases private by default, and requiring a keyword such as `export` to precede the alias.
2. The aliases in the `map` definition might have private access, which may not always be desirable.

These drawbacks might not be compatible with the suggestion from Proposal 3 due to verbosity, and the author would like to open a discussion point.

Implementation status

As the first proposal is still being implemented, the technical difficulties of this proposal have not yet been assessed.

Other thoughts

The author also believes that this is only half of a complete solution for the problems outlined in the initial proposal: it is still necessary to specify a *type-requirement* before using the *alias-declaration*.

Proposal 3

Proposal 3 can be either an extension to Proposal 1 or Proposal 2, and allows a *type-requirement* to be made inline with the *alias-declaration*.

Relevant motivation

One might conclude that we can allow *alias-declarations* to immediately double as a *type-requirement*, but this doesn't work, since the following would not be possible:

```
template <typename T>
requires requires(T t) {
    using initializer_type = initializer_list<value_type_t<T>>;
}
void foo(T t);
void bar()
{
    foo(vector<int>{}); // ill-formed: vector<int>::initializer_type doesn't exist
}
```

The author instead proposes that an *alias-declaration* be prefixed with the `requires` keyword to indicate it is both an alias, and a requirement; otherwise, it is just an alias. For example,

```
template <typename T>
requires requires(T t) {
+   requires using value_type = value_type_t<T>;
    using initializer_type = initializer_list<value_type>;
    requires Constructible<T, initializer_type>();
}
void foo(T t);
void bar()
```



```
{
    foo(vector<int>{}); // ok: vector<int> satisfies the two requirements
    foo(pair<int, int>{}); // ill-formed: pair<int, int>::value_type doesn't exist
}
```

In this example, we see that `vector` still conforms to the requirements outlined by `foo` : namely, `typename value_type_t<T>`, and `Constructible<T, initializer_type>` (for some type `initializer_type`). As stated previously, compatibility between this proposal and the previous proposal may be somewhat difficult.

Implementation status

No assessment has been made regarding technical feasibility at present.

Conclusion

In summary, we have looked at a proposal to allow for alias-declarations in requirement-bodies, a proposal extending the declarations to following functions and classes to reduce duplicate aliasing, and finally, a proposal that combines the aliasing and type-requirement into a single statement to further limit duplicate names.

Although there are some potential issues, the author remains hopeful that they can be ironed out.

Acknowledgements

Casey Carter and Eric Niebler entertained the initial idea and suggested potential alternative ways to implement this proposal, one of which became the actual implementation.

Tom Honermann and Casey also provided a proof reading of the document, and suggested several improvements.